Problem Tracker — Segment 8: Q&A Buffer

8 of 8 · 3 min + as-you-go

Backup reference, not a rehearsed segment — skim once beforehand, don't read from it live

> Given as a reference for "we'll follow your lead and ask questions as we go" — not something to present linearly. Numbers current as of today: 354 problems (239 LeetCode / 115 NeetCode), 59/354 real SM-2 divergences, 3 self-caught bugs to draw from if asked "tell me about a mistake."

Data Model & Mistakes

"Why SQLite and not Postgres from the start?"

Zero-setup, single-user, ~350 rows — Postgres solves a concurrency problem that doesn't exist yet.

"Biggest mistake you've found?"

Three, at different points: the two-file DB split with no cross-file FK; a query-string-in-the-primary-key bug that created 127 duplicate rows; a double-counted scoring discount caught by writing tests. Full stories in segment 4.

"Why a separate catalog/accepted_dates table instead of a status column / delimited string?"

Cramming a second concept onto an existing column or row makes its meaning ambiguous by inspection, and a delimited string can't be queried or counted — a real table keeps each fact independently true.

Algorithm & Sources

"Why not just use SM-2?"

It answers "is this due," not "which top-N across everything" — still needs a ranking step on top regardless.

"Then why build SM-2 at all?"

"I considered X and rejected it" is only real if you can show the comparison — a tested SM-2 plus a real-data replay makes the trade-off demonstrable.

"What happens if the two sources disagree?"

They disagree on coverage, not facts — a problem solved outside NeetCode legitimately shows unsolved there.

Testing & Scaling

"How do you test this app?"

113+ unit tests plus live browser-driven testing — launching the real app and driving it with Playwright to catch what unit tests can't, like a version-incompatible keyword argument or a rendering-scale freeze.

"How would this scale to more users?"

The real wall is SQLite's one-writer-at-a-time model. Migrate to Postgres, split problems into a shared catalog plus a per-user table for the fields that are actually per-user.

General — Prep Your Own Answer

Genuinely personal — have a real ~20-second answer ready, not a memorized line:

"Why did you build this?"

"What are you most proud of in this project?"

Glossary — Fast Recall

Upsert

INSERT ... ON CONFLICT DO UPDATE — insert-or-update as one atomic statement, avoiding a check-then-write race.

Foreign key

Engine-enforced reference from one table to another's primary key — makes "every review points at a real problem" a guarantee, not a hope.

WAL mode

SQLite's Write-Ahead Log — writers append to a separate log file instead of locking the whole DB.

MVCC

Multi-version concurrency control (Postgres) — each transaction sees a consistent snapshot; readers never block writers.

RLS

Row-level security (Postgres) — the DB itself refuses another user's rows, even if app code has a bug.

CAP theorem

Doesn't apply yet — this app has one copy of the data on one machine, no partition to have an opinion about. Becomes relevant the moment this syncs across devices.